

1.1.1. Calculate Momentum

01:32



Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum p is calculated using the formula:

$$p = m \times v$$

where:

- m is the mass of the object (in kilograms).
- v is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Sample Test Cases



Explorer

calculate...



Submit

```
1 mass=float(input())
2 vel=float(input())
3 mom=mass*vel
4 print(f"{mom:.2f}kgm/s")
```

Debugger

Average time
0.005 s
5.50 ms

Maximum time
0.007 s
7.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 7 ms

Debug



Expected output

5.0

10.0

50.00kgm/s

Actual output

5.0

10.0

50.00kgm/s

Test case 2 7 ms



Terminal

Test cases

Prev Reset Submit Next

1.1.2. Conditional Calculation Based on the Number of Digits

01:35

Write a Python program that accepts an integer n as input. Depending on the number of digits in n .

Constraints:

 $1 \leq n \leq 999$

Input Format:

- The input consists of a single integer n .

Output Format:

- If n is a single-digit number, print its square.
- If n is a two-digit number, print its square root (rounded to two decimal places).
- If n is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid".

Sample Test Cases

+

Explorer

condition...

Submit

```
1 import math
2 n=int(input())
3 if n>=1 and n<=9:
4     print(n*n)
5 elif n>=10 and n<=99:
6     print(f"{math.sqrt(n):.2f}")
7 elif n>=100 and n<=999:
8     print(f"{math.cbrt(n):.2f}")
9 else:
10    print("Invalid")
11
```

Average time

0.003 s

3.14 ms



Maximum time

0.005 s

5.00 ms



4 out of 4 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1

5 ms

Debug

Expected output

9

81

Actual output

9

81

Test case 2

3 ms

Test case 3

2 ms

Terminal

Test cases

Prev

Reset

Submit

Next

1.1.3. Days between Two Dates

10:27

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format *YYYY – MM – DD*.
- The second line contains the second date in the format *YYYY – MM – DD*.

Output Format:

- An integer representing the number of days between the given dates.

Note:

- The first date should always be considered earlier than the second date.

Sample Test Cases



Explorer

noOfDay...



Submit

```
1 from datetime import datetime
2 date1=input()
3 date2=input()
4
5 d1=datetime.strptime(date1,"%Y-%m-%d")
6 d2=datetime.strptime(date2,"%Y-%m-%d")
7
8 diff=(d2-d1).days
9 print(diff)
```

Debugger

Average time
0.009 s
9.00 ms



Maximum time
0.022 s
22.00 ms



2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 22 ms

Debug



Expected output

2014-07-02

2014-11-02

123

Actual output

2014-07-02

2014-11-02

123

Test case 2 5 ms



Terminal

Test cases

1.1.4. Reverse a Number

00:33

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

Sample Test Cases

reverseN...

```

1 n=int(input())
2 rev=int(str(n)[::-1])
3 print(rev)

```

Average time: 0.003 s
Maximum time: 0.005 s

3.40 ms 5.00 ms

2 out of 2 shown test case(s) passed
3 out of 3 hidden test case(s) passed

Test case 1 5 ms Debug

Expected output	Actual output
5367	5367
7635	7635

Test case 2 3 ms

Terminal Test cases

1.1.5. Multiplication Table

01:03



Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:

```
<number> x <multiplier> = <result>
```

Note:

- The character "x" is used in the output to represent multiplication.
- Refer to the visible test-cases for more clarity.

Sample Test Cases



Explorer

multiplica...



Submit

```
1 n=int(input())
2 for i in range(1,11):
3     print(f"{n} x {i} = {n*i}")
```

Debugger

Average time

0.004 s

4.25 ms



Maximum time

0.007 s

7.00 ms



2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

7 ms

Debug



Expected output

8

8 x 1 = 8

8 x 2 = 16

8 x 3 = 24

8 x 4 = 32

8 x 5 = 40

Actual output

8

8 x 1 = 8

8 x 2 = 16

8 x 3 = 24

8 x 4 = 32

8 x 5 = 40

Terminal

Test cases

Prev

Reset

Submit

Next

1.2.1. Pass or Fail

32.29

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer n , the number of courses.
- The second input will be n integers representing the marks of the student in each of the n courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Note:

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses

passorFa...

```
1 n=int(input())
2 marks=list(map(int,input().split()))
3 if any(mark<40 for mark in marks):
4     print("Fail")
5 else:
6     avg=sum(marks)/n
7     print(f"Aggregate Percentage: {avg:.2f}")
8
9     if avg>75:
10        print("Grade: Distinction")
11    elif avg>=60:
```

Average time

0.005 s

5.30 ms

Maximum time

0.007 s

7.00 ms

5 out of 5 shown test case(s) passed

5 out of 5 hidden test case(s) passed

Test case 1 6 ms

Debug

Expected output

4

55 65 78 89

Aggregate Percentage: 71.75

Grade: First Division

Actual output

4

55 65 78 89

Aggregate Percentage: 71.75

Grade: First Division

Test case 2 7 ms

Terminal

Test cases

1.2.2. Fibonacci Series

02:11

Write a Python program that uses recursion to print the first n terms of the Fibonacci series.

Input Format:

- A single integer n representing the number of terms to generate.

Output Format:

- A single line containing the first n terms of the Fibonacci sequence, separated by spaces.

Sample Test Cases

+

fibonacci....

Submit

```
1 def fibonacci(n):
2     if n==1:
3         return 0
4     elif n==2:
5         return 1
6     else:
7         return fibonacci(n-1) + fibonacci(n-2)
8
9     n = int(input())
10    for i in range(1, n + 1):
11        print(fibonacci(i), end=" ")
```

Average time
0.008 s
8.00 ms

Maximum time
0.021 s
21.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 5 ms Debug

Expected output

5

0 1 1 2 3

Actual output

5

0 1 1 2 3

Test case 2 2 ms

Terminal

Test cases

1.2.3. Pattern - 1

02:17



Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

Sample Test Cases



rightangl...

Submit

```
1 n=int(input())
2 for i in range(1,n+1):
3     for j in range(i):
4         print("*",end=" ")
5     print()
```

Average time

0.004 s

3.83 ms



Maximum time

0.006 s

6.00 ms



2 out of 2 shown test case(s) passed

4 out of 4 hidden test case(s) passed

Test case 1 4 ms Debug

Expected output

5

* .

* * .

* * * .

* * * * .

* * * * * .

Actual output

5

* .

* * .

* * * .

* * * * .

* * * * * .

Terminal

Test cases

< Prev

Reset

Submit

Next >

1.2.4. Pattern - 2

02:00 A A C P -

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

- Refer to the displayed test cases for the sample pattern.

Sample Test Cases

+

numberP...

Submit

```
1 n=int(input())
2 for i in range(1,n+1):
3     for j in range(1,i+1):
4         print(j,end=" ")
5     print()
```

Average time

0.004 s

4.50 ms

Maximum time

0.006 s

6.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 6 ms

Debug

Test cases

Expected output

5

1

1 2

1 2 3

1 2 3 4

1 2 3 4 5

Actual output

5

1

1 2

1 2 3

1 2 3 4

1 2 3 4 5

Terminal

Test cases

< Prev

Reset

Submit

Next >